

고급머신러닝 보고서

R - CNN

Regions with Convolutional Neural Network

전자공학부 전자공학과
2020144005 김윤성
2024. 10. 22 제출

목차 INDEX

01 R - CNN

- 객체 검출
- Selective Search
- R-CNN의 한계

02 실습 #1)

- R - CNN
- 코드 분석
- 실행 결과
- 분석

03 실습 #2)

- NMS
- R - CNN + NMS
- 코드 분석
- 실행 결과
- 분석

The background is a solid dark blue with several abstract geometric shapes. A large, light blue curved shape is on the right side. A dark blue diagonal band runs from the bottom left towards the top right. A smaller, medium blue curved shape is located in the upper right quadrant.

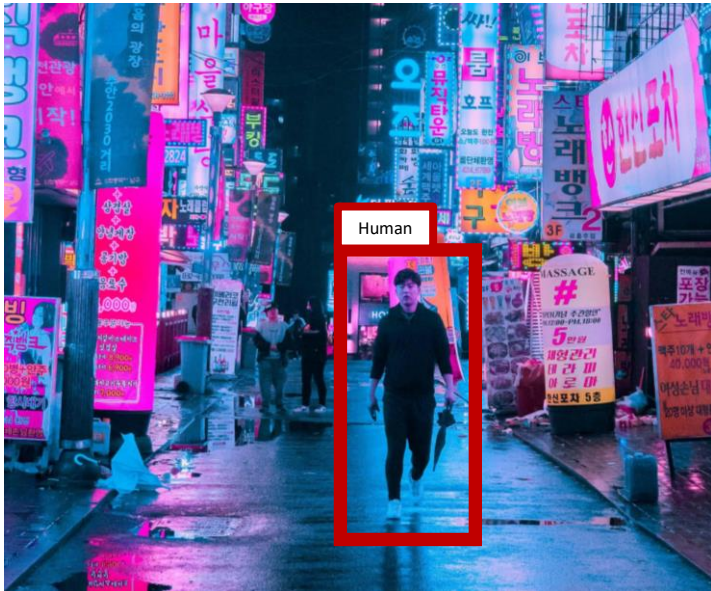
01

R - CNN

Regions with Convolutional Neural Network

객체 검출 Object Detection

- Classification: 이미지에서 어떤 물체가 있는지 처리
- Localization: 그 물체가 이미지에서 어디에 있는지 처리
- Instance Segmentation: 그 물체가 어떤 픽셀에 있는지



Classification + Localization

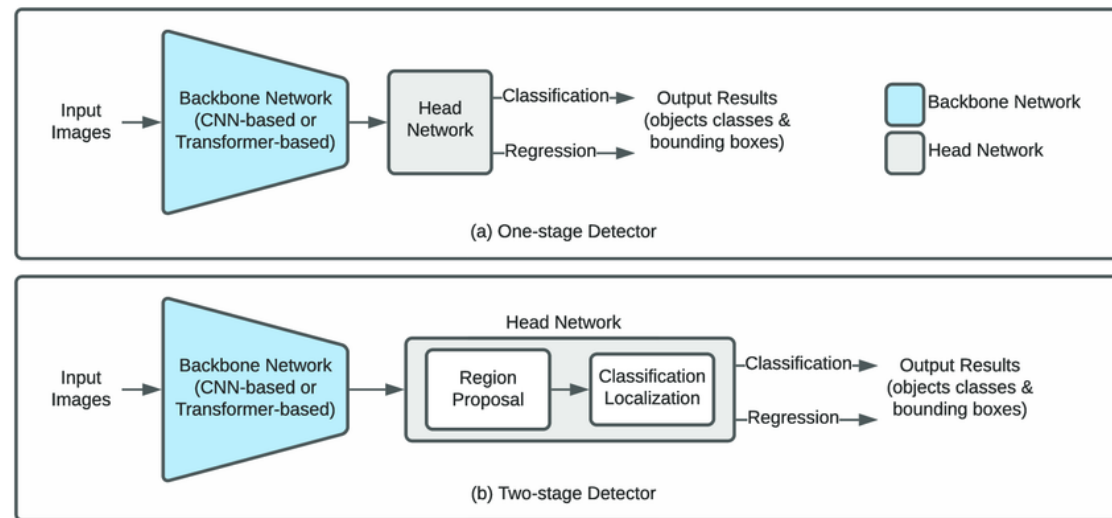


Instance Segmentation

01 R - CNN Regions with Convolutional Neural Network

검출 Detector

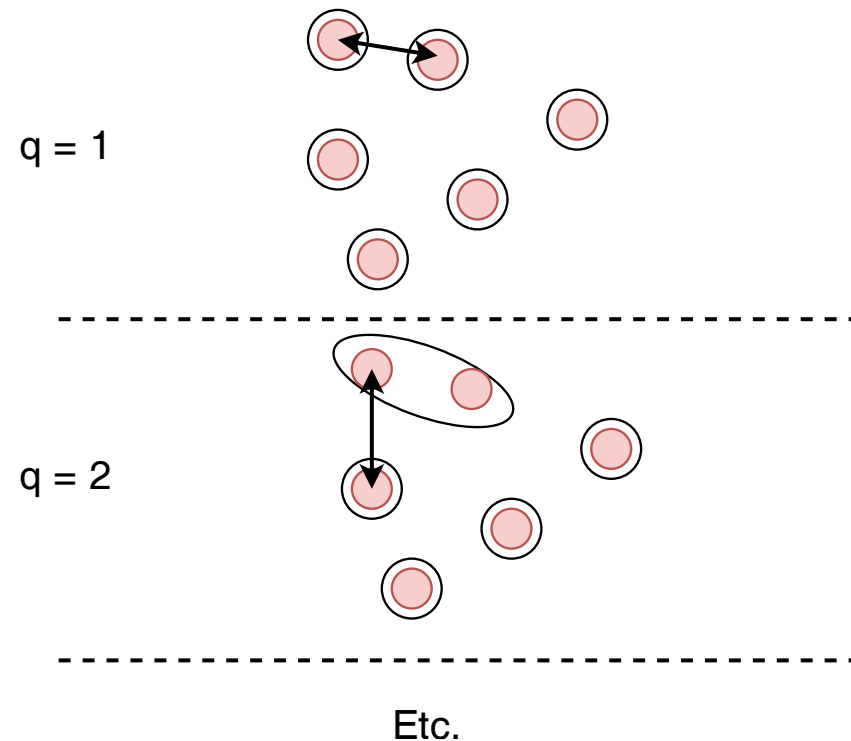
- **Region Proposal:** 물체가 있을것 같은 영역을 일단 Bounding Box로 처리
- **1-Stage Detector:** Region Proposal과 Classification을 동시에 처리
처리 속도가 빠르고, 실시간 처리에 적합
YOLO 같은 알고리즘이 여기 속함
- **2-Stage Detector:** Region Proposal 후 Classification
처리 속도가 느리지만 정확도가 높음
R-CNN, Fast/Faster R-CNN이 여기 속함



Efficient graph-based image segmentation

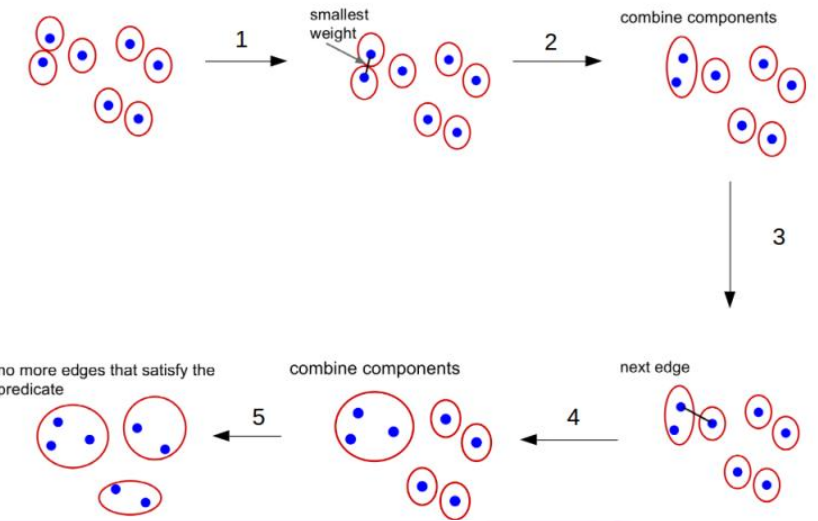
- 이미지의 각 픽셀은 노드로 표현되고, 인접한 노드 사이에는 엣지가 생성되며, 해당 엣지의 가중치는 픽셀 간 유사성에 따라 결정됨
- 초기에는 각 노드를 각각의 cluster로 지정
- 엣지는 모든 픽셀 간 서로 연결할 수도 있으나, 이미지가 커지면 비효율적이기 때문에 k-mean clustering과 유사한 방식으로 군집화
- 군집간의 차가 군집 내의 차보다 큰 경우는 별개의 군집으로 그대로 있고, 그렇지 않은 경우에는 병합한다. (false일 때 병합)

$$D(C_1, C_2) = \begin{cases} \text{true} & \text{if } Dif(C_1, C_2) > MInt(C_1, C_2) \\ \text{false} & \text{otherwise} \end{cases}$$



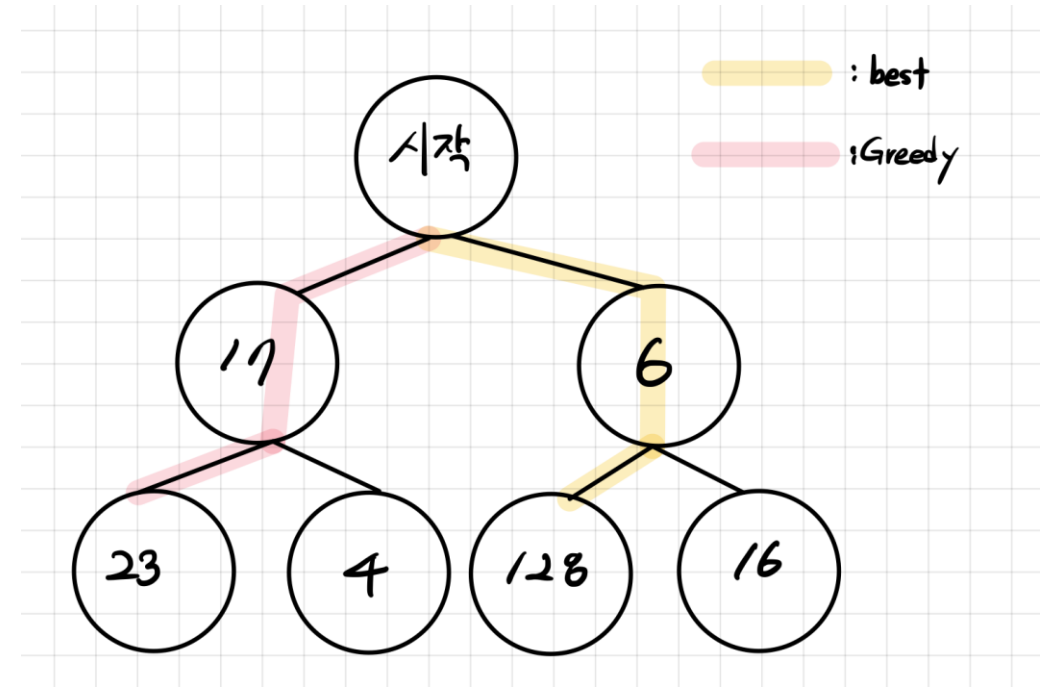
Efficient graph-based image segmentation

- 해당 알고리즘을 반복 시행했을 때의 결과가 Selective Search의 초기 영역이다.



Greedy Algorithm

- 쉽게 말해, 근시안적 선택 알고리즘
현재 상황에서 가장 좋은 결과를 선택하는 알고리즘이다.
- 아까 만든 초기 영역에서, 두 영역간의 유사도 계산 후,
가장 유사한 2개를 1개로 병합 (유사도 계산 시 색, 크기 등 고려)
조건을 만족할때까지 계속 반복



Greedy Algorithm



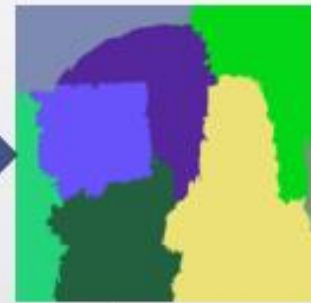
Input Image



After Initial
Segmentation



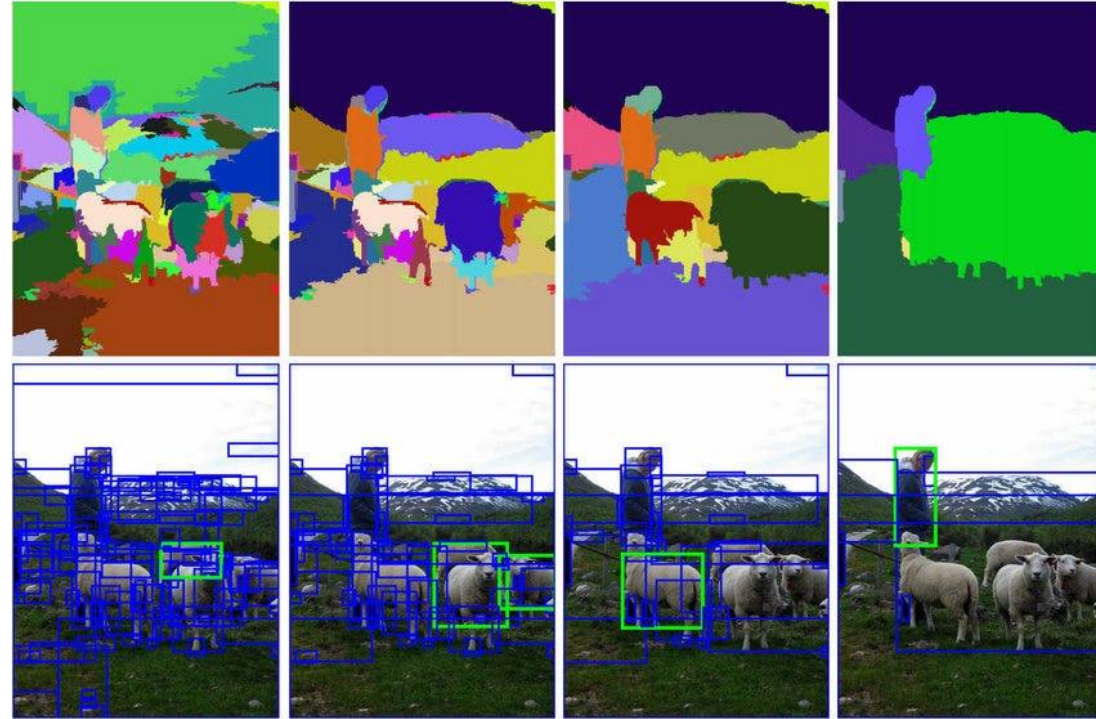
After few
Iterations



After many
iterations

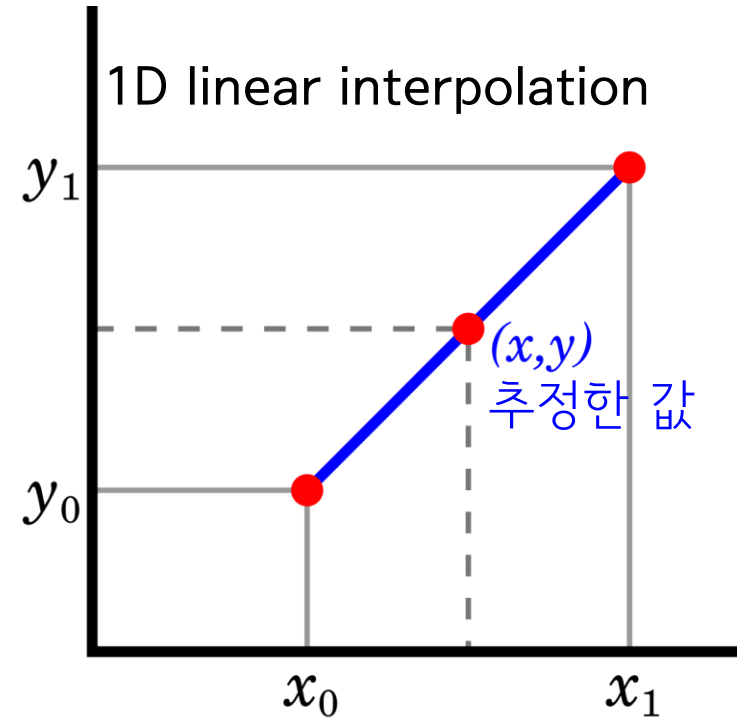
Selective Search

- Exhaustive Search: Brute Force Attack과 유사한 방식이며, 세 글자로 요약하면 노가다이다.
- Segmentation: 이미지에서 유사한 부분을 픽셀별로 레이블화 하는 알고리즘
- Selective Search: Segmentation + Exhaustive Search로, 이미지에서 물체가 있을 것 같은 영역을 Segmentation한다.
- Efficient graph-based image segmentation을 이용하여 초기 영역을 결정한 후, Greedy Algorithm을 활용하여 유사 영역을 병합하고, 그 영역들을 바탕으로 후보 영역들을 추출



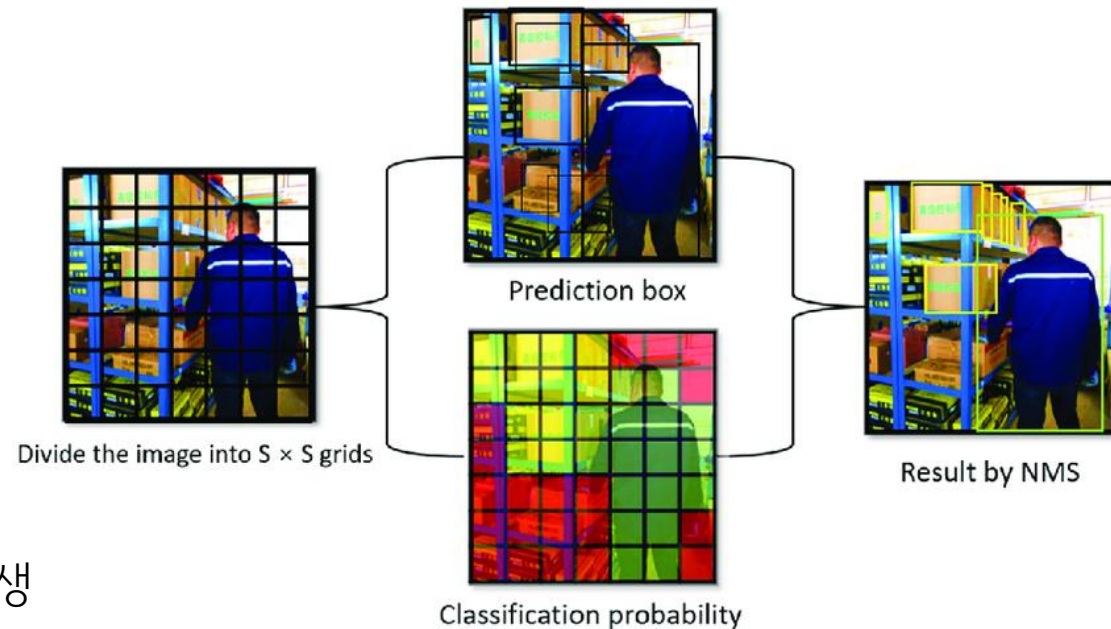
Warp

- Selective Search를 통해 나온 Bounding Box를 특정 크기로 통일시키는 과정
- 값 사이에 값을 추정하는 Interpolation 과정을 포함한다.
- Linear, Polynomial, Spline, Local 등의 Interpolation 기법을 이용한다.
- 고차 Polynomial Interpolation은 Overshoot(튀는 값)현상이 발생해 불안정한 결과가 나올 수 있음



R - CNN의 한계 Limitation of R-CNN

- 2-stage detector: Selective Search 과정을 따로 거치기 때문에 느린 속도
- Selective Search로 도출된 영역만 사용하기 때문에 배경을 객체로 탐지하는 오류 잦음
- 이미지를 변형시키면 분류 성능 저하
- 해당 문제를 해결하기 위해 1-stage detector인 YOLO 탄생



02

실습 #1

R - CNN

실습 #1)

R-CNN 실습 과정을 따라 R-CNN 구현

- 여러 데이터에 대해 R-CNN 실습 과정을 구현 및 결과 분석
- 실패한 예측 및 이상한 결과에 대한 원인 분석

02 실습 #1 코드 분석

```
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential, Model, load_model
import selectivesearch
import os
import cv2
```

필요한 모듈 импорт

```
model_path = 'file_path\\model.h5'
cnn_model = load_model(model_path)
img_dir = 'MNIST\\data\\images\\'
img_list = os.listdir(img_dir)

for i in range(len(img_list)):
    img = cv2.imread(img_dir + img_list[i])
    temp_img = img.copy()

    _, regions = selectivesearch.selective_search(img, scale=800, min_size=1200)
    region_rects = [cand['rect'] for cand in regions]
    region_rects = np.array(region_rects)
```

모델과 이미지를 로드하고, Selective Search 과정을 거친다.

이미지를 불러올 때, 원본을 훼손하지 않게 하기 위해 이미지의 복사본을 만든다.

for문을 활용하여 모든 데이터에 대해 작업하도록 했다.

사용된 모델은 Chap.4에서 쓰인 cifar_model을 MNIST를 처리할 수 있도록 input shape를 수정한 버전이다.

02 실습 #1 코드 분석

```
bbox = []
bboxes = []
for rect in region_rects:
    xmin = rect[0]
    ymin = rect[1]
    xmax = xmin + rect[2]
    ymax = ymin + rect[3]
    bbox = (xmin, ymin, xmax, ymax)
    bboxes += [bbox]

bboxes = np.array(bboxes)
for i, cand in enumerate(bboxes):
    bb = cand
    temp_img = cv2.rectangle(temp_img, (bb[0], bb[1]), (bb[2], bb[3]), color=(0,255,0), thickness=2)
```

Bounding Box를 정의한다.

Selective Search 과정에서 도출된 후보 영역들의 좌표(rects)를 바탕으로 좌표를 계산하고, 녹색 사각형으로 표시한다.

02 실습 #1 코드 분석

```
test_img = []
for i in range(len(bboxes)):
    cropped_img = temp_img[bboxes[i][1]:bboxes[i][3], bboxes[i][0]:bboxes[i][2]]
    warped_img = cv2.resize(cropped_img, (28,28), interpolation=cv2.INTER_LINEAR)
    warped_img_gray = cv2.cvtColor(warped_img, cv2.COLOR_BGR2GRAY)
    warped_img_gray = np.expand_dims(warped_img_gray, axis=-1)
    test_img.append(warped_img_gray)

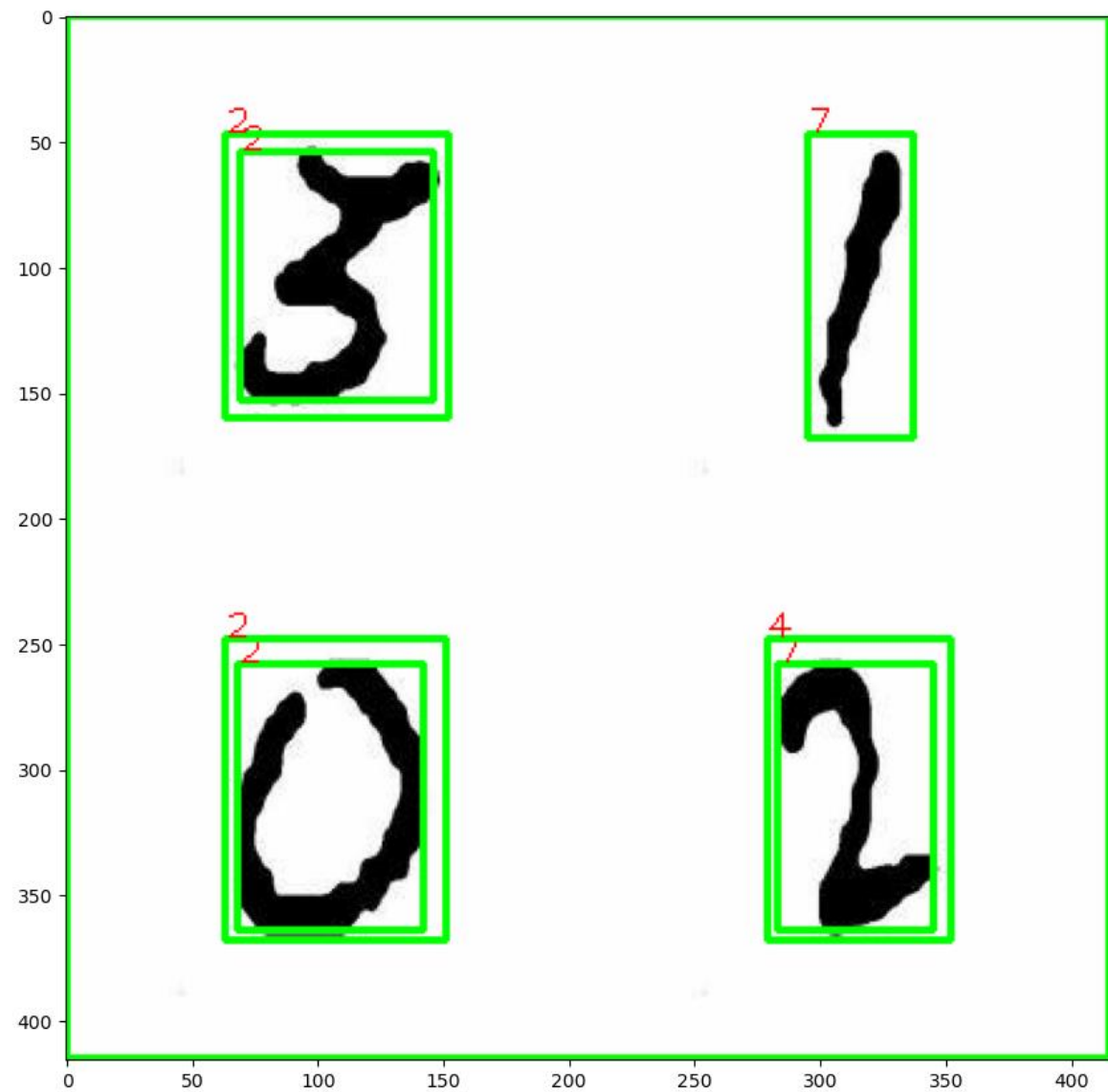
test_img = np.array(test_img)

y_pred_prob = cnn_model.predict(test_img)
y_pred = np.argmax(y_pred_prob, axis=1)

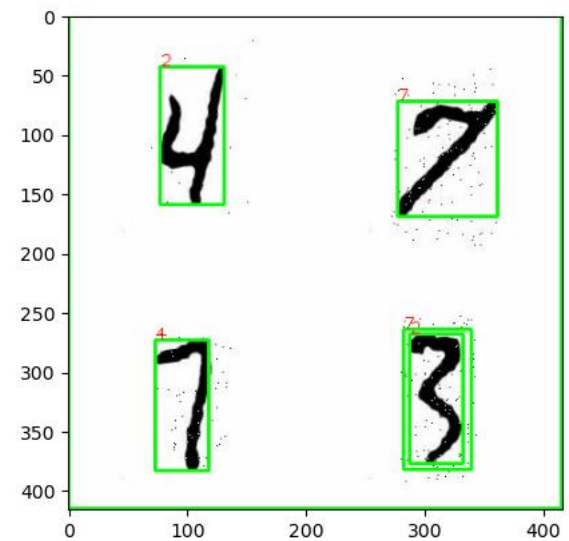
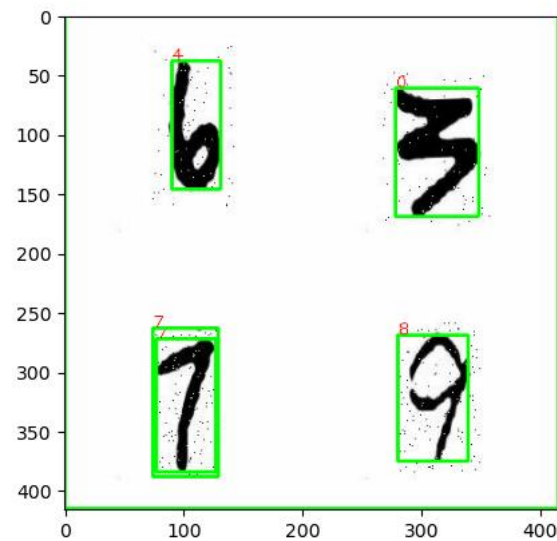
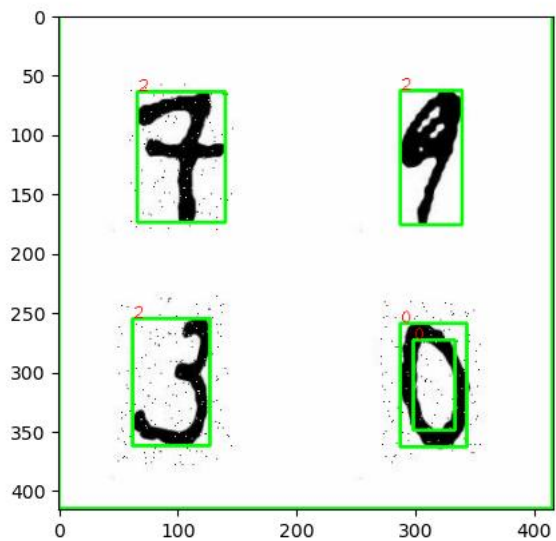
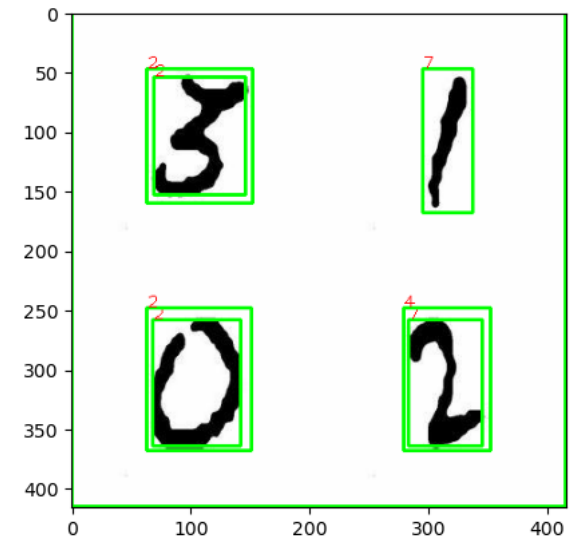
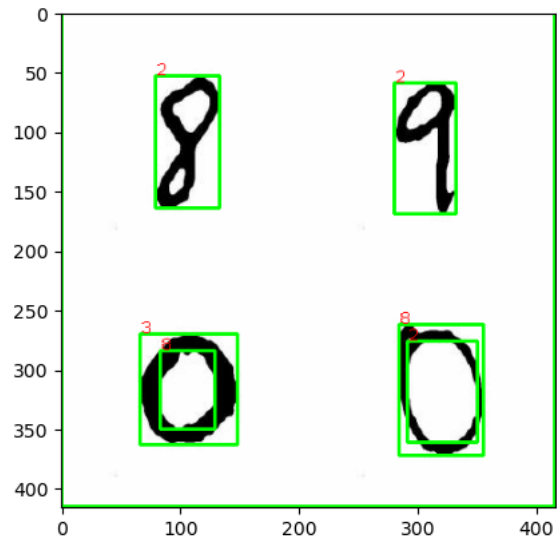
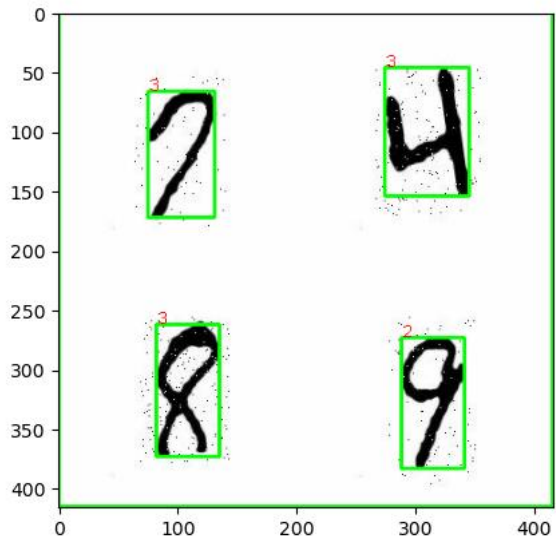
test_img = img.copy()
for i, cands in enumerate(bboxes):
    bb = cands
    temp_img = cv2.rectangle(temp_img, (bb[0],bb[1]), (bb[2],bb[3]), color=(0,255,0), thickness=2)
    temp_img = cv2.putText(temp_img, str(y_pred[i]), (bb[0],bb[1]), 1, 1, color=(255,0,0))
plt.imshow(temp_img)
plt.show()
```

각 후보 영역을 자른 후, (28, 28)의 크기로 resize한다.
영역을 흑백으로 변환한 후, 모델에 집어넣어 가장 높은 확률의 클래스를 y_pred로 추출한다.
그 후, 예측된 클래스 라벨을 빨간 텍스트로 표시한다.

02 실습 #1 실행 결과

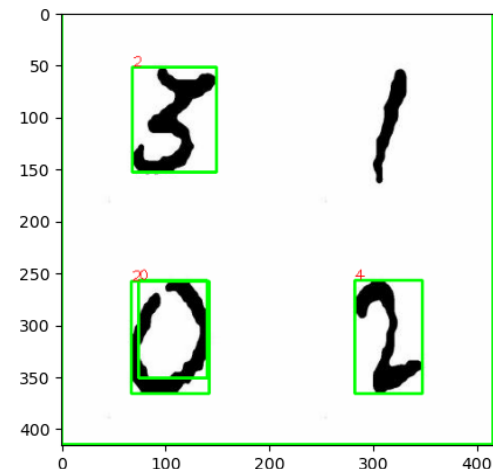
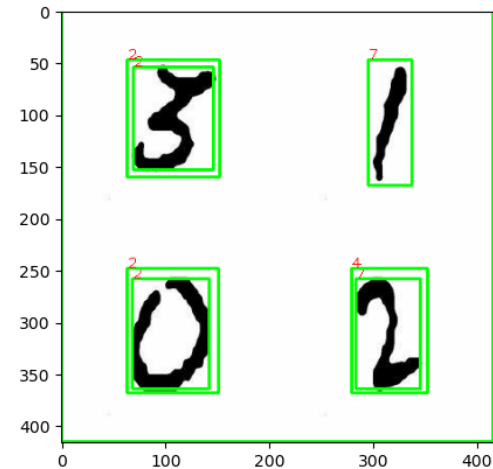


02 실습 #1 실행 결과



02 실습 #1 분석

- MNIST 데이터는 회전 각도가 약간 다르거나, 노이즈가 약간 다르거나 등등 원본 데이터에서 조금씩 변형된 데이터 총 984개로 이루어져 있다.
- 현재 파악된 문제는 아래와 같다.
 - 한 숫자에 두 개의 Bounding Box가 쳐지는 문제
 - > Selective Search가 동일한 객체에 대해 중복된 영역을 생성하기 때문
 - 데이터가 약간만 달라져도 인식을 못하는 문제 (인식을 실패하는 문제)
 - > resize중 Region이 숫자와 일치하지 않는것으로 추정됨
 - 숫자를 이상하게 인식하는 문제
 - > Bounding Box가 숫자를 모두 포함하지 않아 생기는 문제로 추정됨
 - 이미지 전체에 Bounding Box를 치는 문제
 - > 숫자와 관계없는 배경까지 후보 영역에 포함되기 때문



03

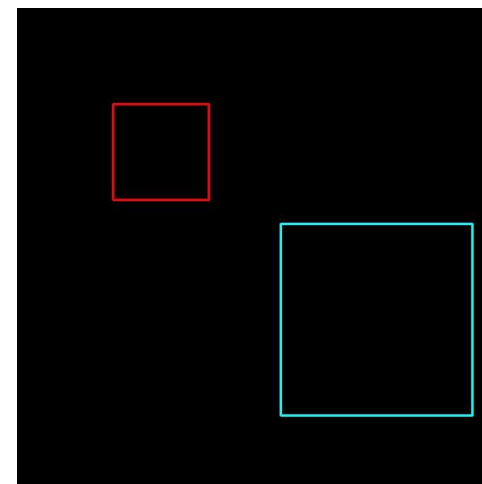
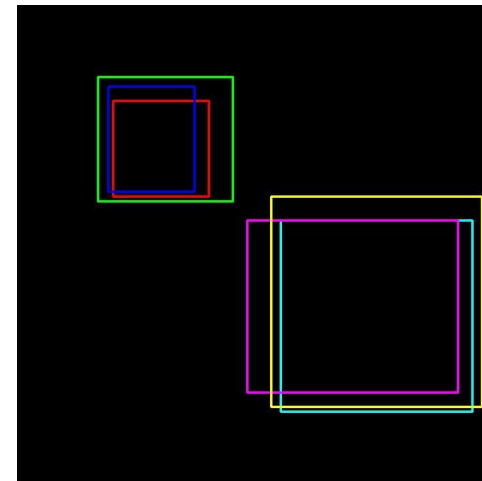
실습 #2

R – CNN with NMS

NMS

Non Maximum Suppression

- 간단히, 최대가 아닌 Bounding Box들을 제거하는 알고리즘이다.
이로 인해 한 객체에 여러 개의 박스가 생기는 상황 방지 가능
- 바운딩박스들을 특정 기준에 따라 정렬
- 가장 점수가 높은 박스와 나머지 박스가 겹치는 범위를 계산한 후,
일정 값보다 높으면 겹친 것으로 간주하여 삭제
- 모든 박스가 처리될 때까지 반복



실습 #2) R-CNN의 문제점을 해결하는 자신만의 아이디어 구현

- R-CNN의 문제점을 해결하는 알고리즘을 구현 하시오.
- • 해결 가능한 문제점
 - ✓ 이미지 전체를 탐지
 - ✓ 객체 예측 실패
 - ✓ 하나의 객체에 여러 Bounding Box
- • 아이디어 예시
 - ✓ Selective Search 시, Bounding Box의 크기가 일정 크기 이상인 경우 해당 Bounding 박스 삭제 or 일정 크기 이하인 경우에만 Bounding Box 저장
→ 이미지 전체 탐지 방지
 - ✓ Warping 시 객체의 왜곡 발생 → Warping 된 이미지를 활용하여 CNN 학습
 - ✓ Bounding Box가 일정 비율 이상 겹치는 경우 하나만 남기고 Box 삭제

03 실습 #2 코드 분석

```
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential, Model, load_model
import selectivesearch
import os
import cv2
```

필요한 모듈 импорт

```
model_path = 'file_path\\model.h5'
cnn_model = load_model(model_path)
img_dir = 'MNIST\\data\\images\\'
img_list = os.listdir(img_dir)
```

이미지를 불러올 때, 원본을 훼손하지 않게 하기 위해 이미지의 복사본을 만든다.

사용된 모델은 Chap.4에서 쓰인 cifar_model을 MNIST를 처리할 수 있도록 input shape를 수정한 버전이다.

03 실습 #2 코드 분석

```
def non_max_suppression(bboxes, overlapThresh=0.5):
    if len(bboxes) == 0:
        return []
    x1 = bboxes[:, 0]
    y1 = bboxes[:, 1]
    x2 = bboxes[:, 2]
    y2 = bboxes[:, 3]
    area = (x2 - x1 + 1) * (y2 - y1 + 1)
    idxs = np.argsort(y2)
    pick = []
    while len(idxs) > 0:
        last = len(idxs) - 1
        i = idxs[last]
        pick.append(i)
        xx1 = np.maximum(x1[i], x1[idxs[:last]])
        yy1 = np.maximum(y1[i], y1[idxs[:last]])
        xx2 = np.minimum(x2[i], x2[idxs[:last]])
        yy2 = np.minimum(y2[i], y2[idxs[:last]])
        w = np.maximum(0, xx2 - xx1 + 1)
        h = np.maximum(0, yy2 - yy1 + 1)
        overlap = (w * h) / area[idxs[:last]]
        idxs = np.delete(idxs, np.concatenate(([last], np.where(overlap > overlapThresh)[0])))
    return bboxes[pick].astype("int")
```

NMS를 시행하는 함수이다.

이 함수를 좀 해체해보도록 하겠다.

03 실습 #2 코드 분석

```
if len(bboxes) == 0:  
    return []
```

만약 바운딩박스가 없으면 빈 리스트 반환

```
x1 = bboxes[:, 0]  
y1 = bboxes[:, 1]  
x2 = bboxes[:, 2]  
y2 = bboxes[:, 3]  
area = (x2 - x1 + 1) * (y2 - y1 + 1)
```

바운딩박스의 각 좌표를 추출한 뒤, 박스의 면적 계산

```
idxs = np.argsort(y2)
```

y2를 기준으로 바운딩박스 정렬, 인덱스 구하기

```
while len(idxs) > 0:  
    last = len(idxs) - 1  
    i = idxs[last]  
    pick.append(i)
```

가장 큰 y2를 가진 바운딩박스를 pick 리스트에 추가

```
xx1 = np.maximum(x1[i], x1[idxs[:last]])  
yy1 = np.maximum(y1[i], y1[idxs[:last]])  
xx2 = np.minimum(x2[i], x2[idxs[:last]])  
yy2 = np.minimum(y2[i], y2[idxs[:last]])  
w = np.maximum(0, xx2 - xx1 + 1)  
h = np.maximum(0, yy2 - yy1 + 1)  
overlap = (w * h) / area[idxs[:last]]
```

선택된 바운딩 박스와 나머지의 겹치는 영역 계산

03 실습 #2 코드 분석

```
idxs = np.delete(idxs, np.concatenate(([last], np.where(overlap > overlapThresh)[0])))
```

만약 overlapThresh (디폴트 0.5)를 넘기면 박스 제거

```
return bboxes[pick].astype("int")
```

최종적으로 선택된 박스들만 반환

03 실습 #2 코드 분석

```
def filter_large_bboxes(bboxes, img_shape, max_ratio=0.5):
    filtered_bboxes = []
    img_area = img_shape[0] * img_shape[1]

    for bbox in bboxes:
        bbox_area = (bbox[2] - bbox[0]) * (bbox[3] - bbox[1])
        if bbox_area / img_area <= max_ratio:
            filtered_bboxes.append(bbox)

    return np.array(filtered_bboxes)
```

이미지 전체에 Bounding Box를 치는 것을 방지하기 위해 추가한 함수이다.
Bounding Box의 영역이 이미지 전체의 max_ratio (디폴트 0.5) 이하인 박스만 남기고 다 삭제한다.

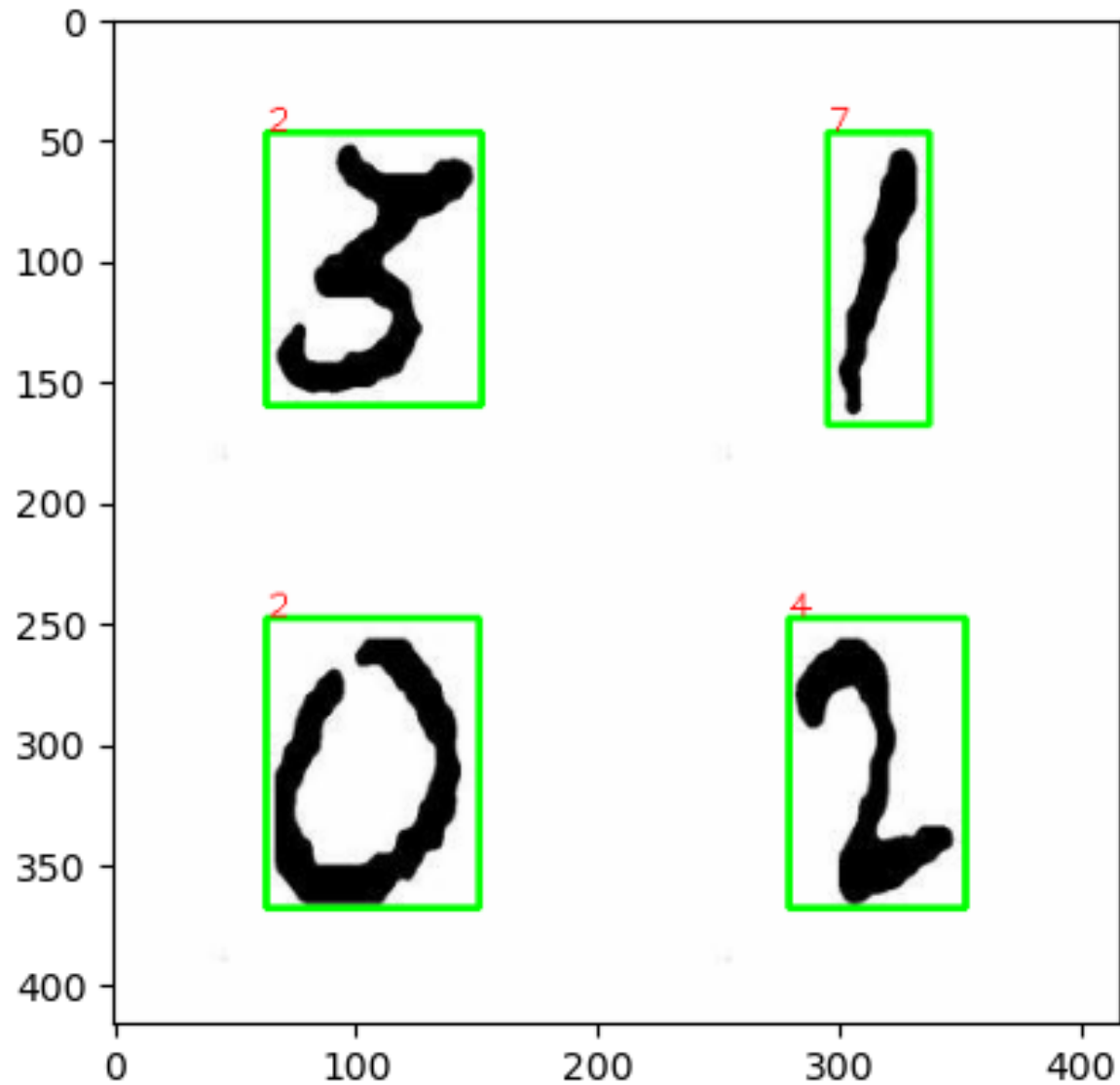
```
for i in range(len(img_list)):
    img = cv2.imread(img_dir + img_list[i])
    temp_img = img.copy()

    bboxes = []
    test_img = []

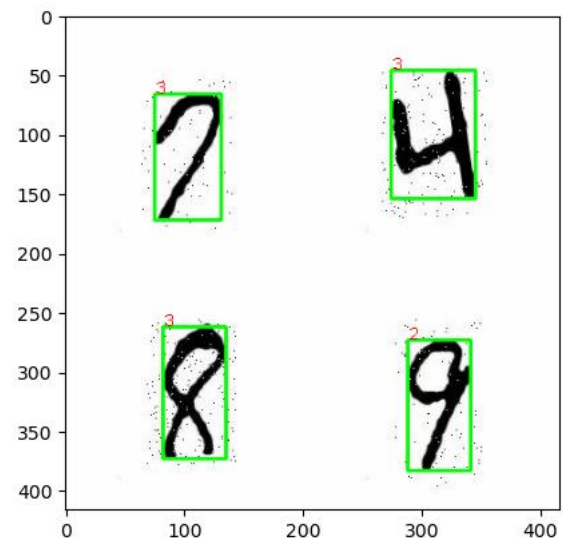
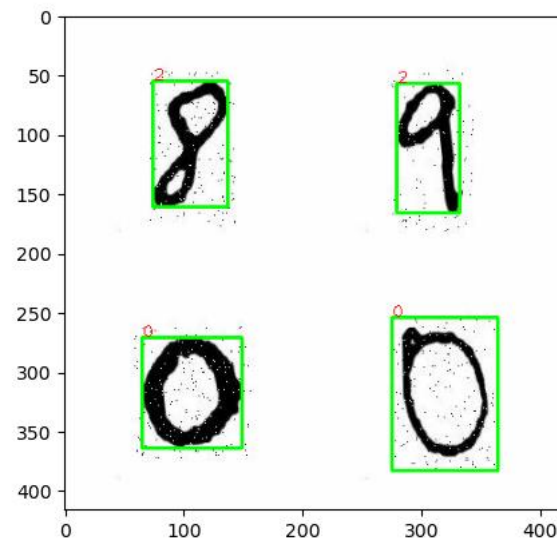
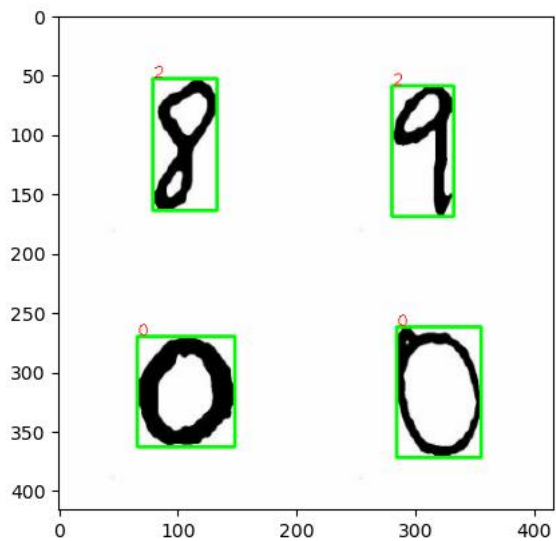
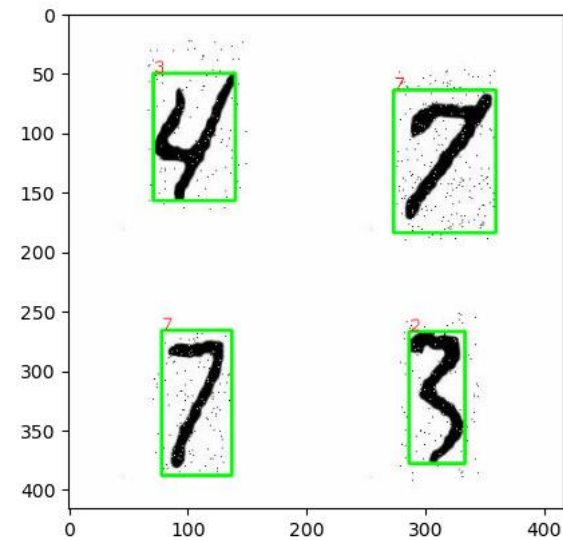
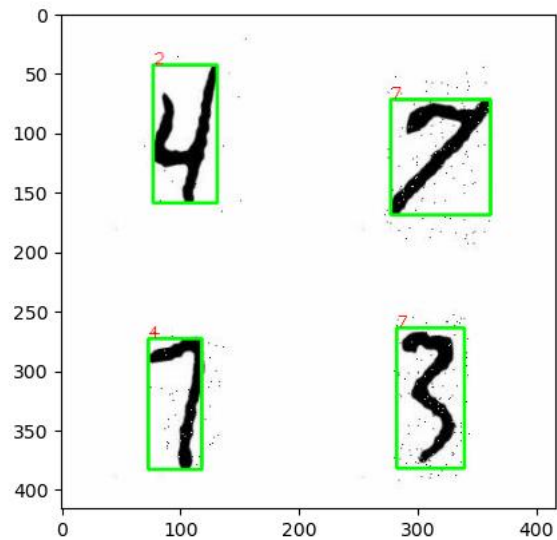
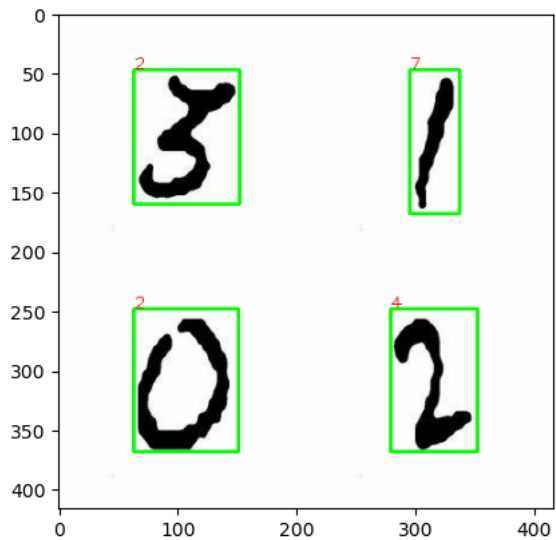
    _, regions = selectivesearch.selective_search(img, scale=800, min_size=1200)
    region_rects = [cand['rect'] for cand in regions]
    region_rects = np.array(region_rects)
```

이하의 코드는 실습 #1과 유사하다. (non_max_suppression(), filter_large_bboxes()만 추가됨)

03 실습 #2 실행 결과

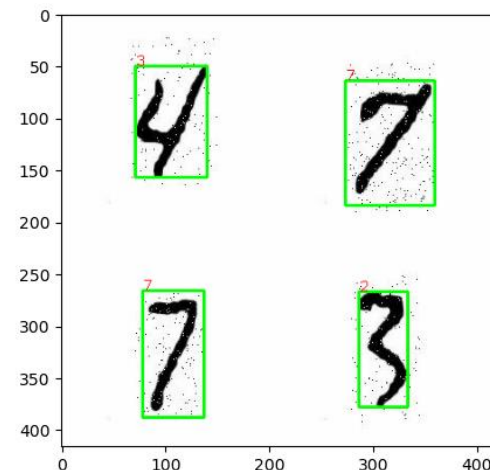
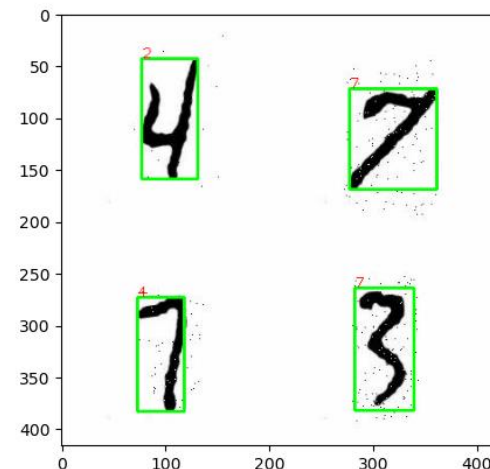


03 실습 #2 실행 결과



03 실습 #2 분석

- Non Maximum Suppresion을 적용한 결과,
한 객체에 여러 개의 Bounding Box가 생기는 현상은 사라졌다.
- 모델이 값을 이상하게 예측하는 경우가 있는데, 이는 resize과정에서
이미지의 정보가 유실되었거나 모델이 충분히 훈련되지 않아 생긴 결과로 추측된다.



고급머신러닝 보고서
감사합니다